

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:
Студент группы ИУ5-34Б
Верзаков Н.В.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Код main.py

```
class Catalog:
    def __init__(self, id_catalog, name):
        self.id_catalog = id_catalog
        self.name = name

class File:
    def __init__(self, id_file, name, size, id_catalog):
        self.id_file = id_file
        self.name = name
        self.size = size
        self.id_catalog = id_catalog

class CatalogFile:
    def __init__(self, id_file, id_catalog):
        self.id_file = id_file
        self.id_catalog = id_catalog

catalogs = [
    Catalog(1, "Документы"),
    Catalog(2, "Изображения"),
    Catalog(3, "Видео")
]

files = [
    File(1, "Анализ.txt", 1250, 1),
    File(2, "Архив.zip", 8500, 1),
    File(3, "Фото.png", 3200, 2),
    File(4, "Видео.mp4", 7000, 3),
    File(5, "Приложение.apk", 900, 1)
]

catalog_files = [
    CatalogFile(1, 1),
    CatalogFile(2, 1),
    CatalogFile(3, 2),
    CatalogFile(3, 1),
    CatalogFile(4, 3),
    CatalogFile(5, 1),
]

def get_files_starting_with(letter, files, catalogs):
    result = []
    for f in files:
        if f.name.startswith(letter):
            for c in catalogs:
                if f.id_catalog == c.id_catalog:
                    result.append((f.name, c.name))
    return result

def get_catalogs_with_min_file_size(files, catalogs):
```

```

result = []
for c in catalogs:
    sizes = [f.size for f in files if f.id_catalog == c.id_catalog]
    if sizes:
        result.append((c.name, min(sizes)))
return sorted(result, key=lambda x: x[1])

def get_files_and_catalogs_many_to_many(files, catalogs, catalog_files):
    result = []
    for cf in catalog_files:
        for f in files:
            for c in catalogs:
                if cf.id_file == f.id_file and cf.id_catalog == c.id_catalog:
                    result.append((f.name, c.name))
    return sorted(result, key=lambda x: x[0])

```

Код test.py

```

import unittest
from main import (Catalog, File, CatalogFile, get_files_starting_with,
                  get_catalogs_with_min_file_size,
                  get_files_and_catalogs_many_to_many)

class TestFileCatalogLogic(unittest.TestCase):
    def setUp(self):
        self.catalogs = [
            Catalog(1, "Документы"),
            Catalog(2, "Изображения"),
            Catalog(3, "Видео")
        ]

        self.files = [
            File(1, "Анализ.txt", 1250, 1),
            File(2, "Архив.zip", 8500, 1),
            File(3, "Фото.png", 3200, 2),
            File(4, "Видео.mp4", 7000, 3),
            File(5, "Приложение.apk", 900, 1)
        ]

        self.catalog_files = [
            CatalogFile(1, 1),
            CatalogFile(2, 1),
            CatalogFile(3, 2),
            CatalogFile(3, 1),
            CatalogFile(4, 3),
            CatalogFile(5, 1),
        ]

    def test_files_starting_with_A(self):
        result = get_files_starting_with("A", self.files, self.catalogs)

```

```

        expected = [
            ("Анализ.txt", "Документы"),
            ("Архив.zip", "Документы")
        ]
        self.assertEqual(result, expected)

    def test_min_file_size_by_catalog(self):
        result = get_catalogs_with_min_file_size(self.files, self.catalogs)
        expected = [
            ("Документы", 900),
            ("Изображения", 3200),
            ("Видео", 7000)
        ]
        self.assertEqual(result, expected)

    def test_many_to_many_relationship(self):
        result = get_files_and_catalogs_many_to_many(
            self.files, self.catalogs, self.catalog_files
        )
        expected = [
            ("Анализ.txt", "Документы"),
            ("Архив.zip", "Документы"),
            ("Видео.mp4", "Видео"),
            ("Приложение.арк", "Документы"),
            ("Фото.png", "Изображения"),
            ("Фото.png", "Документы")
        ]
        self.assertEqual(result, expected)

if __name__ == "__main__":
    unittest.main()

```

Результат выполнения программы

```

(.venv) rowa@rowas-MacBook-Pro VERZAKOV_PCPL_Labs % /Users/rowa/Documents/Codes/VERZAKOV_PCPL_Labs/.venv/bin/python /Users/rowa/Documents/Codes/VERZAKOV_PCPL_Labs/test.py
...
-----
Ran 3 tests in 0.000s

OK
(.venv) rowa@rowas-MacBook-Pro VERZAKOV_PCPL_Labs %

```